

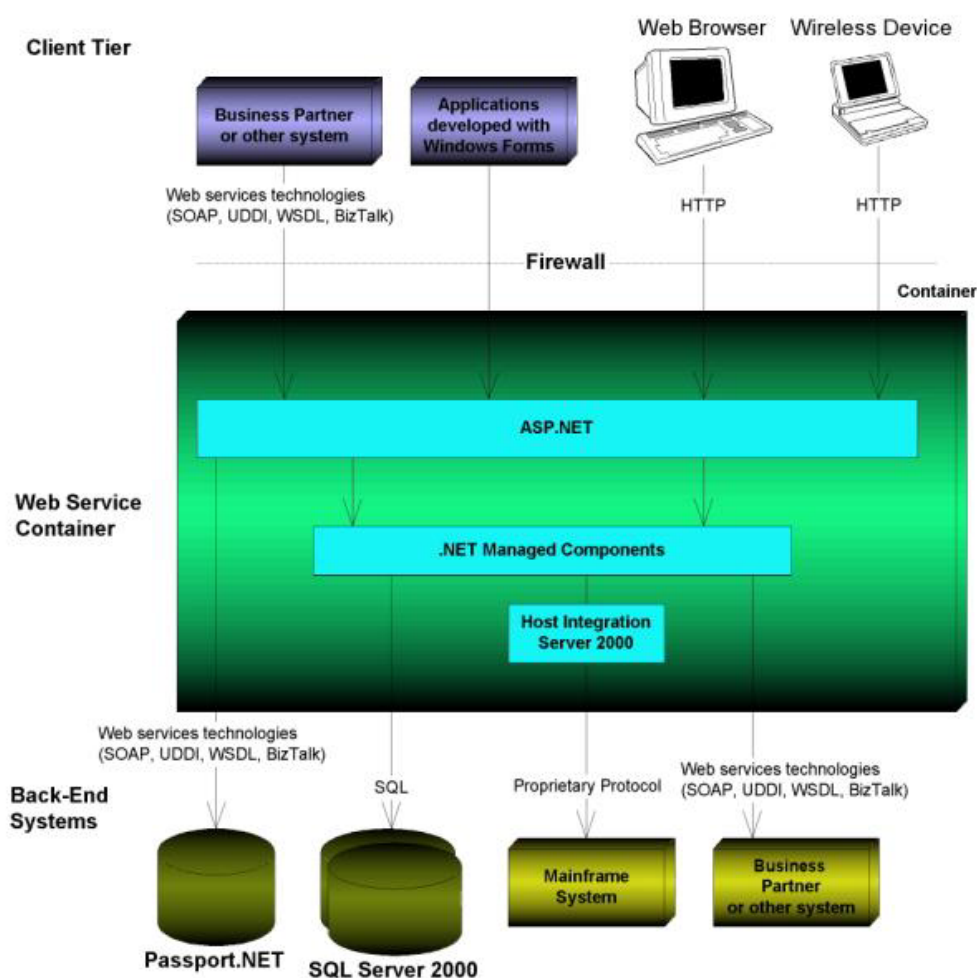
บทที่ 5

เปรียบเทียบ .NET เว็บเซอร์วิส และจาวาเว็บเซอร์วิส

ในบทนี้เราจะกล่าวถึงเทคโนโลยีในการพัฒนาโซลูชันจากทั้งสองค่าย ตั้งแต่แพลตฟอร์มในการพัฒนา และในส่วนของการสร้างเว็บเซอร์วิส

5.1 แพลตฟอร์มการพัฒนา

5.1.1 แพลตฟอร์มของ .NET



รูปที่ 5 - 1 แพลตฟอร์มการพัฒนาระบบภายใต้เทคโนโลยี .NET

.NET แอปพลิเคชัน

ถูกคิดฟอยอยู่ภายในคอนเทนเนอร์ ที่มีเซอร์วิสที่จำเป็นให้บริการสำหรับเอนเตอร์ไพรส์ แอปพลิเคชัน เช่น ทรานแซกชัน ความปลอดภัย และ เมสเสจจิงเซอร์วิส

ชั้นของธุรกิจ

ของ.NET แอปพลิเคชันถูกสร้างโดยใช้ .NET เมนาจคอมโพเนนต์(managed component) ชั้นนี้ทำใน ส่วนของการประมวลผลทางธุรกิจและการจัดการข้อมูล การติดต่อไปยังฐานข้อมูลโดยผ่าน ADO.NET มีการ ติดต่อกับระบบเก่า(legacy system) โดยผ่าน Microsoft Host Integration Server 2000 เช่นติดต่อกับ COM Transaction Integrator (COM TI) หรือติดต่อคู่ค้าทางธุรกิจผ่านเว็บเซอร์วิส

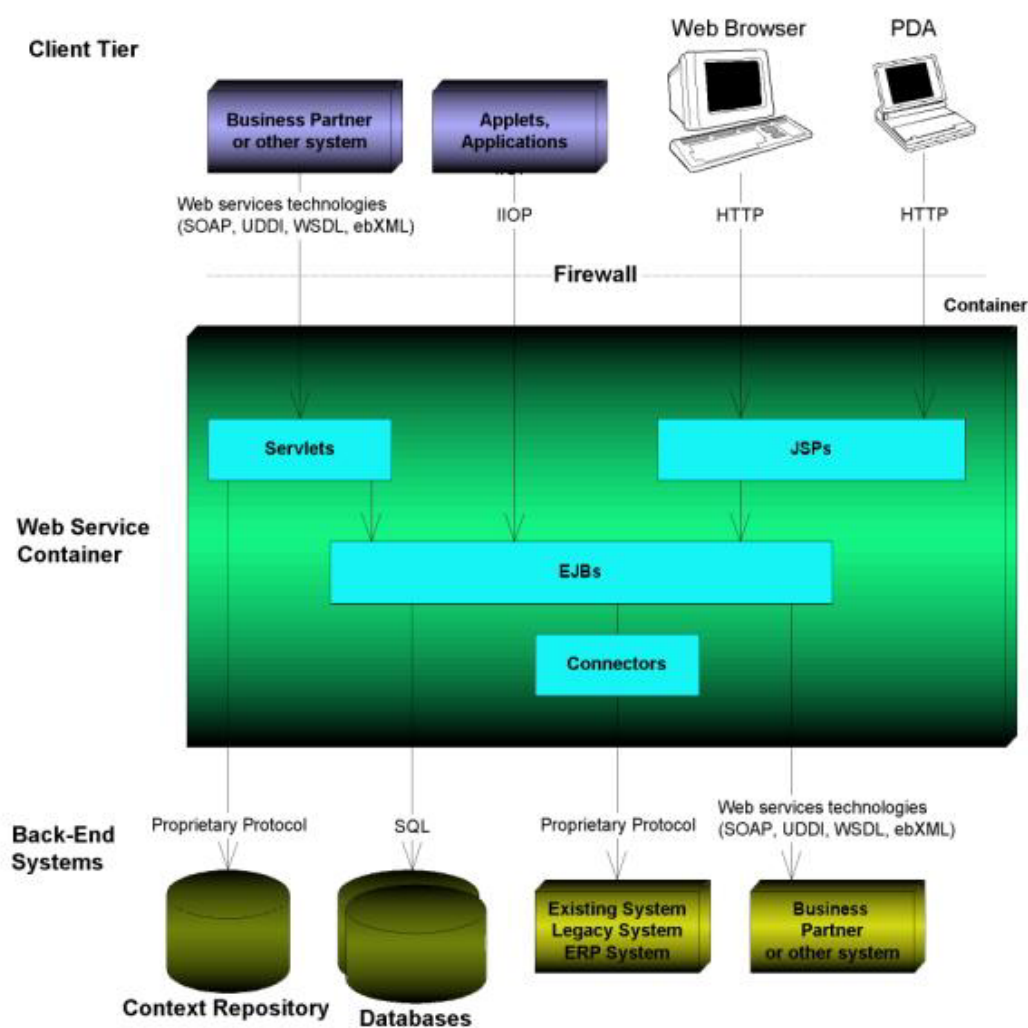
คู่ค้าทางธุรกิจ

สามารถใช้ .NET แอปพลิเคชันติดต่อโดยผ่านเว็บเซอร์วิส

ไคลเอนต์ที่เรียกใช้งาน

ก็เป็นได้ทั้งผ่านเว็บ ASP.NET ซึ่งให้ผลได้ในรูปของ HTML, XHTML, WML และรูปแบบของ วินโดว์ฟอร์ม

5.1.2 แพลตฟอร์มของจาวา



รูปที่ 5 - 2 แพลตฟอร์มการพัฒนาระบบภายใต้เทคโนโลยี J2EE

J2EE แอปพลิเคชัน

ถูกตีฟลอยอยู่ภายในคอนเทนเนอร์ ที่มีเซอร์วิสที่จำเป็นให้บริการสำหรับเอนเตอร์ไพรส์ แอปพลิเคชัน เช่น ทรานแซกชัน ความปลอดภัย
ชั้นของธุรกิจ

ในแอปพลิเคชันขนาดใหญ่ถูกสร้างโดยใช้ Enterprise JavaBeans (EJB) คอมโพเนนต์ ติดต่อฐานข้อมูลโดย JDBC หรือ SQL/J หรือติดต่อกับระบบเดิมผ่าน Java Connector Architecture (JCA) ติดต่อผ่านเว็บเซอร์วิสโดยใช้ จาวา API สำหรับXML (JAX APIs)

ลูกค้าทางธุรกิจ

ติดต่อผ่านเว็บเซอร์วิส ตัวเซิร์ฟเลท(servlet) เป็นตัวคอยรับการร้องขอที่เข้ามาและส่งผลลัพธ์กลับไปโดยใช้ JAX APIs เป็นตัวจัดการเกี่ยวกับเว็บเซอร์วิส

โคลเอนต์ที่เรียกใช้งาน

นอกเหนือจากการผ่านเว็บ และเว็บเซอร์วิสแล้ว ยังมีแอปเพลตที่สามารถจะติดต่อกับจาวาบินผ่านยังโปรโตคอล Internet Inter-ORB Protocol (IIOP) ได้ด้วย

5.2 เปรียบเทียบเทคโนโลยีของทั้งสองค่าย

เริ่มจากแสดงให้เห็นถึงความคล้ายคลึงกันของทั้งสองแพลตฟอร์มดังตารางข้างล่าง และจะทำการพิจารณาเปรียบเทียบให้เห็นในแต่ละหัวข้อ

Feature	J2EE	.NET
Type of technology	Standard	Product
Middleware Vendors	30+	Microsoft
Interpreter	JRE	CLR
Dynamic Web Pages	JSP	ASP.NET
Middle-Tier Components	EJB	.NET Managed Components
Database access	JDBC SQL/J	ADO.NET
SOAP, WSDL, UDDI	Yes	Yes
Implicit middleware (load-balancing, etc)	Yes	Yes

ตารางที่ 5 – 1 ตารางแสดงการสนับสนุนการทำงานด้านต่างๆของ 2 แพลตฟอร์ม

5.2.1 Vendor Solution

ทางด้านของ J2EE นั้นเป็นสเปกที่ออกมาโดยมีเวนเดอร์ (vendor) หลายเจ้าที่นำเอามาตรฐานเหล่านั้นไปพัฒนาผลิตภัณฑ์ (product) ออกมารองรับ รวมทั้งมีเครื่องมือให้ใช้ค่อนข้างหลากหลาย ข้อดีคือมีตัวเลือกในการพัฒนาโซลูชันค่อนข้างหลากหลาย แต่ข้อเสียก็มาจากข้อดีของมันนั่นเองคือ ความไม่เข้ากันของแต่ละผลิตภัณฑ์ ไม่สามารถที่จะย้ายโค้ดโปรแกรมที่พัฒนาข้ามผลิตภัณฑ์ได้อย่างครบถ้วนซึ่งการพัฒนาระบบจริงนั้นเหมาะกับการเลือกใช้ผลิตภัณฑ์จากค่ายเดียวกันมากกว่า

ทางฝั่ง .NET ออกผลิตภัณฑ์ออกมาน่า เนื่องจากเป็นผู้ออกสเปกและพัฒนาเองทั้งหมด รวมทั้งระบบที่ทำการสนับสนุนการทำงานส่วนมากก็เป็นของไมโครซอฟท์เอง

5.2.2 การสนับสนุนระบบเดิม

ในระบบเดิมที่มีการใช้งานโดยหลากหลายภาษาหลายระบบในส่วนของจาวานั้นก็มีหลากหลายทางในการทำให้สามารถใช้งานร่วมกันกับระบบเดิมเหล่านั้นได้

- ใช้ Java Message Service (JMS) เชื่อมกับระบบเก่า
- ใช้เว็บเซอร์วิสในการอินทิเกรตกับระบบต่างๆ
- ติดต่อผ่าน CORBA

รวมทั้งยังมี J2EE Connector Architecture (JCA) ทำหน้าที่เป็น ตัวติดต่อกับทรัพยากรอื่นๆ เช่น SAP R/3 ,CICS/COBOL และอื่นๆ

ฝั่ง .NET มีตัว Host Integration Server 2000 เป็นตัวช่วยจัดการเกี่ยวกับทรานแซกชันไปยังระบบเมนเฟรม หรือตัว Microsoft Message Queue (MSMQ) ในการทำงานร่วมกันกับระบบเดิมที่ใช้ IBM MQSeries และสุดท้ายคือตัว BizTalk Server2000 ทำการอินทิเกรตระบบบนโปรโตคอลของ B2B

5.2.3 การยอมรับทางการตลาด

ทางด้านจาวามีมากกว่า 50 บริษัทที่ให้การยอมรับในแพลตฟอร์ม

อีกด้านนั้นทีมการตลาดของไมโครซอฟท์ทำงานด้านนี้ได้ค่อนข้างดี รวมทั้งวินโดวส์ก็มีผู้ใช้อยู่มากมาย

รวมทั้งทางด้านไมโครซอฟท์ประกาศตัวเกี่ยวกับเรื่องเว็บเซอร์วิสมาก่อนจาวา ในข้อนี้สรุปได้ว่าไมโครซอฟท์มีการทำการตลาดที่ดีกว่า

5.2.4 ภาษาที่รองรับ

จาวามีจุดเด่นที่เป็นภาษากลางที่สามารถนำไปใช้งานได้หลากหลายแพลตฟอร์ม

.NET มีจุดเด่นตรงที่รองรับได้หลายภาษาที่เป็นไปตามข้อกำหนดของ .NET เฟรมเวิร์ก โดยข้อดีของมันคือ ภายใต้อุปกรณ์เดียวกันสามารถพัฒนาได้จากหลายภาษา แต่ข้อเสียก็เยอะเช่นกัน เช่น ความเสี่ยงของระบบใหม่นี้ที่ทางไมโครซอฟท์พัฒนาขึ้นมา ของใหม่มัธยมมีความเสี่ยงเกิดขึ้น

- การดูแลรักษาระบบ เนื่องจากเขียนด้วยหลากหลายภาษา ถ้าบุคคลผู้เชี่ยวชาญในบางภาษาลาออกไป ทำให้คนอื่น ๆ ที่ดูแลลำบากในการทำความเข้าใจโค้ดภาษานั้นๆ

- การสร้างองค์ความรู้ การใช้งานหลากหลายภาษาส่งผลให้โปรแกรมเมอร์ไม่สามารถแชร์ประสบการณ์หลายๆอย่างในการพัฒนาแก่กันได้

ในการพัฒนาระบบนั้นก็ต้องพิจารณาให้เหมาะสม รวมถึงคำนึงถึงผลกระทบของการตัดสินใจด้วย

5.2.5 การสนับสนุนแพลตฟอร์มเดิม

J2EE ไม่มีปัญหาเรื่องการเชื่อมต่อกับระบบเดิมมากนัก แต่ตัว JCA และ เว็บเซอร์วิสก็ยังต้องการการเขียนโค้ดใหม่เพิ่มเติม ถึงแม้ว่าไมโครซอฟท์ .NET ได้รับการพัฒนามาจาก MTS และ COM+ แต่อย่างไรก็ตามรูปแบบการพัฒนาก็เปลี่ยนไปโดยสิ้นเชิง มีคอมมอนรันไทม์(CLR)เข้ามาจัดการ มีการจัดการเกี่ยวกับคอมโพเนนต์ที่เปลี่ยนไป ทำให้อาจต้องมีการเขียนโค้ดใหม่เกือบ 60% เพื่อให้สามารถใช้ประโยชน์จากรันไทม์เอนเวอร์อนเมนต์ใหม่ได้อย่างเต็มที่

5.2.6 การทำงานข้ามแพลตฟอร์ม (Portability)

J2EE เป็นมาตรฐานสำหรับหลายๆแพลตฟอร์ม และรองรับการอิมพลีเมนต์ได้หลากหลาย แต่ถึงอย่างไรก็ตาม ยังมีปัญหาเรื่องของการนำโค้ดไปใช้ใหม่กับต่างผลิตภัณฑ์ เนื่องจากแต่ละผลิตภัณฑ์ก็ยังมีเพิ่มความสามารถในผลิตภัณฑ์ของตัวเองด้วยที่ไม่เป็นไปตามมาตรฐาน J2EE ทางชันเองก็มีซอฟต์แวร์ที่ให้การทดสอบว่าผลิตภัณฑ์เป็นไปตามมาตรฐาน J2EE ไหม

ถึงวันนี้ ตัวผลิตภัณฑ์ที่รองรับการใช้งาน .NET ก็มาจากไมโครซอฟท์แห่งเดียวจึงไม่มีปัญหาส่วนนี้ แต่มีการพัฒนาเฟรมเวิร์กบนลินุกซ์ด้วยแต่ยังไม่สมบูรณ์นักการพัฒนาคลาสไลบรารี มีการทดสอบการรันโค้ดโปรแกรมง่ายข้ามจาก วินโดวส์ไปยังลินุกซ์ซึ่งก็ได้ผลถูกต้อง

ถึงแม้ว่าไมโครซอฟท์จะรับปากถึงการทำให้ .NET สามารถใช้งานข้ามระบบได้ก็ตาม แต่จากที่ผ่านมาเช่น COM คอมโพเนนต์ ไมโครซอฟท์ก็สามารถทำให้ใช้ได้บนระบบปฏิบัติการอื่นเช่นกัน แต่ไม่มีการพัฒนาเซอร์วิสอื่นๆขึ้นมารองรับการใช้งานเลย ดังนั้นในส่วนของ .NET เราก็ต้องติดตามกันต่อไปว่าจะเป็นอย่างไร

5.2.7 เครื่องมือที่ใช้ในการพัฒนา

จาวามีเครื่องมือให้เลือกใช้ค่อนข้างเยอะ เช่น Forte, Visual Cafe, Visaul Age, JBuilder

ไมโครซอฟท์ ใช้เครื่องมือ Visual Studio.NET ในการพัฒนาซึ่งมีเอนไวรอนเมนต์การใช้งานคล้ายกับ Visual Studio ตัวเดิมซึ่งโปรแกรมเมอร์คุ้นเคยอยู่แล้ว รวมทั้งเครื่องมือของไมโครซอฟท์เองก็มีประสิทธิภาพในการพัฒนาค่อนข้างมาก

5.2.8 การสนับสนุนการใช้งานเว็บเซอร์วิส

J2EE สนับสนุนการใช้งานเว็บเซอร์วิสโดยใช้ Java APIs สำหรับการแปลภาษา XML แต่อย่างไรก็ตามในส่วนนี้ยังอยู่ระหว่างการพัฒนา ยังต้องใช้ปรับแต่งเขียนโค้ดเพิ่มเติมด้วยตัวเองเพื่อให้เป็นเว็บเซอร์วิสได้ ผลิตภัณฑ์ของแต่ละแห่งก็มีการพัฒนาไลบรารีในส่วนนี้เพิ่มเติมขึ้นเองเพื่อให้การใช้งานเป็นไปอย่างอัตโนมัติและรวดเร็ว ซึ่งส่งผลต่อการผูกติดซอร์สโค้ดกับระบบ

เฟรมเวิร์กของ .NET ถูกพัฒนามาเพื่อสนับสนุนเว็บเซอร์วิสอย่างเต็มตัวตั้งแต่แรกเริ่มอยู่แล้ว

ไม่ว่าอย่างไรก็ตามมาตรฐานของเว็บเซอร์วิสยังต้องพัฒนาอีกยาวไกล เช่นเรื่องของการจัดการ ทรานแซกชัน เป็นต้น ดังนั้นทั้งสองค่ายก็จะต้องเร่งออกผลิตภัณฑ์มารองรับในส่วนนี้ด้วย

5.2.9 การขยายการรองรับการใช้งานของระบบ(Scalability)

การเพิ่มประสิทธิภาพของระบบโดยทั่วไปเราสามารถที่จะเพิ่มได้การเลือกใช้อาร์ดแวร์ใหม่ๆที่มีประสิทธิภาพ ทาง J2EE สามารถใช้งานได้หลากหลายแพลตฟอร์มซึ่งแพลตฟอร์มนั้นก็สามารเลือกใช้โพรเซสเซอร์ได้หลายตัวด้วย

ทาง .NET นั้นใช้งานบนวินโดวส์เท่านั้นจึงมีข้อจำกัดทางด้านการสนับสนุนโพรเซสเซอร์

5.2.10 สรุปการเลือกใช้งาน

ทั้งสองแพลตฟอร์มสามารถนำไปสู่จุดมุ่งหมายที่เดียวกันได้ การที่จะเลือกใช้ผลิตภัณฑ์จากค่ายใดก็ต้องพิจารณาจากหลากหลายองค์ประกอบได้แก่ นักพัฒนาในองค์กรของคุณมีความถนัดระบบใด , ระบบเดิมที่ใช้งาน, ความสัมพันธ์กับบริษัทผู้ผลิต, และลูกค้าของบริษัทคุณ

5.3 เปรียบเทียบจาวาและ .NET

มีความพยายามเอาจาวาและ .NET มาเปรียบเทียบกัน โดยมีการเขียนโปรแกรมเลียนแบบแซมเปิลแอปพลิเคชัน (Sample Application) ตัวหนึ่ง ชื่อ เพ็ทสโตร (Petstore) ของจาวาใช้ชื่อว่า เพ็ทช็อป (Petshop) แล้วนำมาวัดประสิทธิภาพกัน ผลออกมาว่า .NET ให้ประสิทธิภาพในการทำงานรวดเร็วกว่าเพ็ทสโตรที่พัฒนาด้วยจาวาถึง 17 เท่า ซึ่งคุณสามารถไปดาวน์โหลดโค้ดได้ฟรี การเปรียบเทียบครั้งนี้ได้รับการประชาสัมพันธ์กันขนานใหญ่ แต่ก็มีโปรแกรมเมอร์ออกมาวิจารณ์กันมาก ถึงความเป็นไปได้ที่อาจจะมีการลำเอียงในการเปรียบเทียบ โดยประเด็นที่มีการโต้แย้งหลักๆ ดังนี้

1.เพ็ทสโตร (Petstore) ถูกออกแบบมาบนพื้นฐานของการแสดงคุณลักษณะของจาวาเป็นหลัก ไม่ได้เน้นเรื่องของความเร็ว การที่เพ็ทช็อปจะมาโจมตีในส่วนของคุณเร็วและเหมารวมไปหมดว่า .NET เหนือกว่านั้น ไม่น่าจะถูกต้องนัก

2. เพ็ทช็อป (Pet Shop) ใช้สโตรโพรซีเคอร์ในฐานะข้อมูลในการทำงาน ซึ่งแน่นอน ย่อมให้ความเร็วในการทำงานเหนือกว่าเพ็ทสโตรอยู่แล้ว และจะไม่สามารถนำไปทำงานบนฐานข้อมูลระบบอื่น ได้ทันที

เหมือนพีทสโตร เพราะสโตรโพชิเตอร์ไม่เหมือนกัน ถ้าจะเปรียบเทียบกันอย่างเป็นธรรมก็ต้องเขียนโปรแกรมติดต่อฐานข้อมูลโดยไม่ใช้สโตรโพชิเตอร์

3. จำนวนบรรทัดของโปรแกรมนั้นพีทช้อปใช้จำนวนบรรทัดเพียง 1 ใน 3 ของพีทสโตร ส่วนหนึ่งเป็นเพราะโครงสร้างภาษา C# ที่พัฒนามาให้กระชับกว่าอยู่แล้ว แต่อีกส่วนหนึ่งก็เกิดจากการเขียนโปรแกรมโดยมีจำนวนบรรทัดของ Error Exception น้อยกว่า

4. การทดสอบนั้น ทำบนแพลตฟอร์มของวินโดวส์ ซึ่งแน่นอนว่า .NET ย่อมจะทำงานได้เป็นอย่างดีกับผลิตภัณฑ์จากบริษัทเดียวกัน แต่ถ้านำไปทดสอบบนระบบอื่นๆ เช่นยูนิกซ์ (UNIX) อาจจะไม่ให้ผลแตกต่างกันมากถึงขนาดนี้

5. มีการใช้แคช (Cache) จำนวนมากกับเพจ (Page) ที่พัฒนาบนพีทช้อป ซึ่งแน่นอน ย่อมทำให้พีทช้อปทำงานได้เร็วขึ้นอยู่แล้ว ขณะที่พีทสโตรไม่ได้เน้นในเรื่องนี้

6. ประเด็นอื่นๆ มีอีกมากมาย เช่น การวางโมเดล (Model) ที่ไม่เหมือนกัน ความเสถียร

5.4 การพัฒนาเว็บเซอร์วิสจากทั้งสองแพลตฟอร์ม

5.4.1 การเรียกใช้งานข้ามแพลตฟอร์ม

จากโปรโตคอล SOAP ที่ใช้ในการติดต่อสื่อสารเว็บเซอร์วิสมีการอธิบายการทำงานแค่เพียงสามส่วน ให้มีการเรียกใช้งานข้ามระบบได้เท่านั้น โดยแลกเปลี่ยนบริการข้อมูลต่างๆในรูปของ XML จากการเรียกใช้งานจริงก็สามารถทำการเรียกใช้เมธอดได้อย่างถูกต้อง

5.4.2 ปัญหาที่พบ

ดังที่กล่าวไว้ในหัวข้อที่แล้วว่า สามารถเรียกเปลี่ยนเซอร์วิสข้อมูลได้เพียงเบื้องต้นเท่านั้น จากการพัฒนาที่จะให้ทั้งสองค่าย มีการใช้เซสชันระหว่างกัน ไม่สามารถทำได้ เพราะเป็นเพียงการพัฒนาในระบบของตัวเองเท่านั้น

การใช้งานในธุรกิจ ปัญหาเรื่องของการทำ two phrase commit การใช้งานเว็บเซอร์วิสไม่สามารถเรียกสนับสนุนการทำงานในส่วนนี้ได้ เนื่องจากโปรโตคอล ภาษา XML ที่ใช้ ไม่ได้รองรับการจัดการทรานแซกชันเลย ยังต้องรอมาตรฐานที่คาดว่าจะออกมาในประมาณปี 2547

ความปลอดภัยในการรับส่งข้อมูลข้ามเครือข่ายก็เป็นปัญหาสำคัญในทางธุรกิจเช่นกัน โดยปัจจุบันการใช้งานที่มีให้ใช้คือ การใช้การพิสูจน์ตัว (Authentication) ผ่านเว็บเซิร์ฟเวอร์ของระบบตนเอง รวมถึงการปรับแต่งระบบให้ใช้โปรโตคอล SSL แต่ก็ทำให้ประสิทธิภาพของระบบลดลงไปด้วย มาตรฐานของความปลอดภัยในเว็บเซอร์วิสที่จะออกมาได้ไม่นาน ออกโดยองค์กร W3C คือมาตรฐาน WS-Security ทางด้าน .NET นั้นออกปลั๊กอินของ ASP.NET เพื่อให้ใช้งานตามมาตรฐานได้ ส่วนฝั่ง J2EE นั้น BEA Web Logic ในเวอร์ชัน 2 ถึงจะมีการสนับสนุนซึ่งอยู่ระหว่างการพัฒนา

ทางด้านจาวายังมีปัญหาที่เกิดขึ้นอีกอย่างคือ การพอร์ตหรือนำโค้ดที่เขียนเพื่อดีพลอยบนเว็บเซิร์ฟเวอร์ตัวหนึ่งไปใช้บนเว็บเซิร์ฟเวอร์ของบริษัทอื่นนั้นไม่สามารถทำได้ เนื่องจากการผูกติดกับไลบรารีของเว็บเซิร์ฟเวอร์ของแต่ละบริษัทด้วย

ข้อมูลเปรียบเทียบ	.NET	Java
1. โครงสร้างภาษา การจัดการหน่วยความจำ JIT เครื่องเสมือน(Virtual Machine) มีหลักการคล้ายๆกัน	แนวคิดแบบ OOP ลอกแบบมาจากจาวา C++ เพิ่มเติมแนวคิดบางประการจากจาวา เช่น เมตาดต้า (Meta Data) มี Intermediate Language(IL) เป็นภาษากลางทำงานบน Common Language Runtime (CLR)	ต้นแบบมาจาก C/C++ มี CORBA IDL และ ORB ช่วยทำให้โค้ดหลายๆภาษาใช้ออบเจกต์ (Object) ร่วมกันได้ ทำงานบน J2EE แต่ไม่ใกล้ชิดกันเหมือนกับ IL ทำงานกับ CLR บน .NET
2. ภาษาที่สนับสนุน	C#,VB.NET เป็นหลัก ภาษาอื่นๆที่สามารถสนับสนุน แต่อาจไม่สะดวกในการใช้งาน เช่น C++,COBOL,Python,Delphi,J#	จาวาเป็นหลัก สามารถใช้ภาษาอื่นได้บ้าง แต่ไม่ดี ต้องการเครื่องมืออื่นเพิ่มเติม เช่น C++,C
3. ราคา	Visual Studio .NET ซึ่งมีราคาแพงและฟรี สำหรับ SDK และ Web Matrix ซึ่งเป็นเครื่องมือพื้นฐานในการพัฒนาที่ไม่แก่นัก	มีเครื่องมือฟรีมากมายแจกจ่าย เครื่องมือที่เก่งกว่าได้แก่ J-Builder ราคาถูกกว่า .NET
4. ความเร็วในการพัฒนา ในงานขนาดเท่ากัน	เร็วกว่าจาวา (ประมาณ 2-5 เท่ากับงานที่เท่ากัน โดยเฉพาะงานเว็บหรือฐานข้อมูล) เพราะเครื่องมือที่มีมาให้เลือก และอำนวยความสะดวกได้มาก	
5. ความเร็วในการทำงานของแอปพลิเคชัน	เร็วกว่า 17 เท่าตามการเปรียบเทียบแอปพลิเคชันที่คล้ายกัน Petstore/Petshop	น่าจะช้ากว่า แต่ไม่น่าจะถึงกับที่ถูกกล่าวอ้างในการเปรียบเทียบระหว่าง PetStore/PetShop
6. การทำงานข้ามแพลตฟอร์ม	- ทำงานได้เฉพาะระบบวินโดวส์ แต่ในอนาคต จะสนับสนุนระบบอื่น เช่น ลินุกซ์ (Linux) - สามารถทำงานข้ามแพลตฟอร์ม	ทำงานได้หลากหลายแทบทุกแพลตฟอร์ม

	ได้ ถ้าเขียนโปรแกรมในลักษณะของเว็บแอปพลิเคชัน (Web Application) ASP.NET แต่การแสดงผลและการทำงานบางอย่าง เช่น Validation Control ก็ไม่สมบูรณ์ 100% บนบราวเซอร์อื่น	
7. คลาสที่สนับสนุน	.NET Framework SDK- มีหลากหลายกว่า 1400 คลาส ที่พัฒนาโดยไมโครซอฟท์ ระบบค่อนข้างผูกขาดบนไมโครซอฟท์	จาวาคอร์เอพีไอ (Java core API) เป็นระบบเปิด มีพัฒนาโค้ดเปิด (Open source) และแจกฟรีมากมาย
8. ความเข้ากันได้กับ Application/Solution อื่นๆ	เข้ากันได้ดีกับผลิตภัณฑ์ของไมโครซอฟท์เอง เช่น ไมโครซอฟท์ออฟฟิศ (MS Office) Exchange, MS SQL	ได้รับการยอมรับจากระบบที่หลากหลายกว่า เช่น IBM , Notes , SAP
9. ขนาดที่เหมาะสมของโครงการและการลงทุน	เหมาะสำหรับโครงการระยะสั้น-กลาง ขนาด ระยะเวลาสั้น-กลาง เช่นเดียวกัน เหมาะกับโครงการขนาดเล็กที่ต้องแข่งขันกันด้านราคา หรือทำเป็นแอปพลิเคชันสำเร็จรูป	เหมาะสมกับโครงการขนาดกลาง-ใหญ่ ระยะเวลาโครงการยาวนานกว่า และมีรายละเอียดข้อกำหนดทางเทคนิค และระบบค่อนข้างมาก
10. การเปิดกว้างทางเทคโนโลยี	เทคโนโลยีค่อนข้างอยู่ในวงจำกัด ข้อมูลเชิงลึกไม่เปิดเผย โครงการในลักษณะโค้ดเปิดค่อนข้างน้อย โค้ดตัวอย่างฟรี หาดูได้ยากกว่า	ค่อนข้างเปิดเผยอย่างกว้างขวาง มีโครงการโค้ดเปิด จำนวนมาก แจกจ่ายให้โค้ดดิ้งที่ค่อนข้างสมบูรณ์ แบบให้ดาวน์โหลดฟรี
11. การสนับสนุนเกี่ยวกับเว็บ	เก่งมากสำหรับงานบนเว็บ มีเทคโนโลยี ASP.NET เขียนเว็บแอปพลิเคชันได้ง่าย และคล้ายคลึงกับการเขียนโปรแกรมบนวินโดวส์ทั่วไป มีเครื่องมือที่เก่งด้านเว็บเซอร์วิส (Webservices) ,SOAP มีคลาสสำเร็จที่สร้างเอกสาร XML ให้เลย โดยใช้เพียง ADO.NET	จาวาเซิร์ฟเวอร์เพจ (Java Server Pages) เครื่องมือในการพัฒนายังค่อนข้างพื้นๆ ต้องเข้าใจ HTML Tag ต่างๆเอง เครื่องมือยังไม่เก่งมากนักเกี่ยวกับงานเว็บเซอร์วิส ต้องเขียนโปรแกรมเพื่อจัดการกับเว็บเซอร์วิสเองเป็นส่วนใหญ่ โดยใช้ JDBC, EJB, JMS, Java XML Libraries

12. สนับสนุนกับวินโดวส์ฟอร์ม	ใช้เครื่องมือพัฒนา Visual Studio.NET ในการพัฒนา	ใช้จาวาสวิงส์ (Java Swing) ซึ่งได้รับการสนับสนุนจากเครื่องมือพัฒนาจาวาที่หลากหลาย
13. Inter-Operatibility	เน้นการทำงานข้ามภาษา	เน้นการทำงานแพลตฟอร์ม

ตารางที่ 5 – 2 สรุปข้อเปรียบเทียบระหว่างสองแพลตฟอร์ม